

MCP + llama.cpp + Qwen2.5 Local Agent Demo

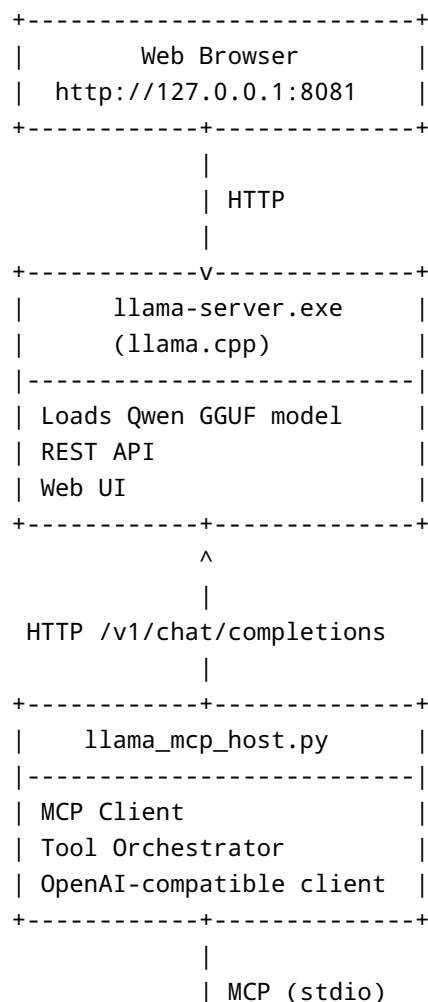
Overview

This project demonstrates how to build a **fully local AI agent** using:

- **llama.cpp** as the inference engine
- **Qwen2.5-1.5B-Instruct (GGUF)** as the LLM
- **MCP (Model Context Protocol)** for tool execution
- **Python** as the orchestration layer

Everything runs locally on a Windows PC without requiring any cloud LLM.

Architecture



```

      |
+-----v-----+
| calculator_server.py |
+-----+
| multiply(a,b)         |
| add(a,b)              |
+-----+

```

Components

Component	Purpose
llama.cpp	Local inference framework
llama-server.exe	Hosts the LLM as an HTTP server
Qwen2.5-1.5B-Instruct GGUF	Language model
llama_mcp_host.py	MCP host/orchestrator
calculator_server.py	MCP server exposing tools

Project Structure

```

mcp-llama-demo/
|
├─ calculator_server.py
├─ client.py
├─ llama_mcp_host.py
├─
└─ README.md

```

Model:

```

I:\technology\models
qwen2.5-1.5b-instruct-q5_k_m.gguf

```

llama.cpp:

```
I:\technology\llama.cpp
```

Step 1 – Install llama.cpp

Download the Windows CUDA build.

Example:

```
llama-b10054-bin-win-cuda-12.4-x64.zip
```

Extract into

```
I:\technology\llama.cpp
```

Step 2 – Verify GPU

```
.\llama-cli.exe --list-devices
```

Example

```
CUDA0  
Quadro P2200
```

Step 3 – Download the model

Example:

```
hf download ^  
  Qwen/Qwen2.5-1.5B-Instruct-GGUF ^  
  qwen2.5-1.5b-instruct-q5_k_m.gguf ^  
  --local-dir I:\technology\models
```

Step 4 – Start llama-server

```
cd I:\technology\llama.cpp

.\llama-server.exe ^
  -m "I:\technology\models\qwen2.5-1.5b-instruct-q5_k_m.gguf" ^
  --host 127.0.0.1 ^
  --port 8081 ^
  --ctx-size 2048 ^
  --jinja ^
  -ngl 99
```

Step 5 – Verify Web UI

Open

```
http://127.0.0.1:8081/
```

You should see the llama.cpp chat interface.

Step 6 – Verify REST API

List models

```
Invoke-WebRequest http://127.0.0.1:8081/v1/models
```

Expected

```
Qwen2.5-1.5B-Instruct
```

Test chat completion

```
$body = @{
  model = "local-model"
  messages = @(
```

```
@{
    role = "user"
    content = "Say hello."
}
)
temperature = 0
stream = $false
} | ConvertTo-Json -Depth 10

$response = Invoke-RestMethod `
    -Uri http://127.0.0.1:8081/v1/chat/completions `
    -Method Post `
    -ContentType "application/json" `
    -Body $body

$response.choices[0].message.content
```

Expected

Hello!

Step 7 – Run the MCP Host

```
cd I:\technology\mcp-llama-demo

python llama_mcp_host.py
```

Example

Ask a question:

What is 25 multiplied by 17?

Output

```
Tools discovered from MCP server

multiply

add
```

```
LLM requested MCP tool

multiply

Arguments

{
  "a":25,
  "b":17
}

MCP server returned

425

Final LLM response

25 multiplied by 17 is 425.
```

Message Flow

```
User
|
▼
llama_mcp_host.py
|
| HTTP
▼
llama-server.exe
|
| decides to call tool
▼
llama_mcp_host.py
|
| MCP
▼
calculator_server.py
|
| execute multiply()
▼
425
|
```

▼
llama_mcp_host.py
|
| HTTP
▼
llama-server.exe
|
▼
Final Answer

Responsibilities

llama.cpp

The inference framework.

Provides

- model loading
- token generation
- GPU execution
- tokenizer

llama-server.exe

Application built on llama.cpp.

Provides

- REST API

POST /v1/chat/completions

- Web UI

http://127.0.0.1:8081

- Hosts the model
-

llama_mcp_host.py

Acts as the AI Agent.

Responsibilities

- discovers MCP tools
 - converts MCP schema to OpenAI tool schema
 - sends tool schemas to the LLM
 - receives tool calls
 - invokes MCP tools
 - returns tool results
 - asks the LLM for the final response
-

calculator_server.py

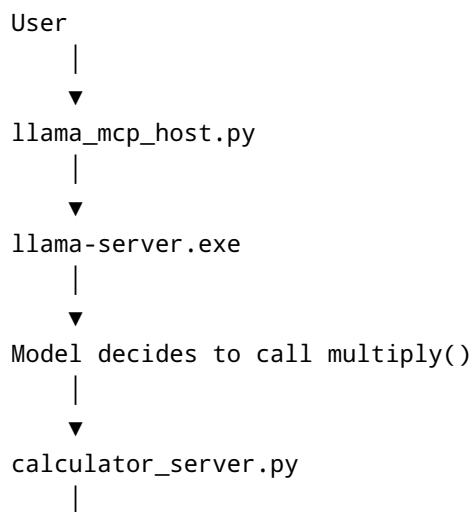
Provides tools through MCP.

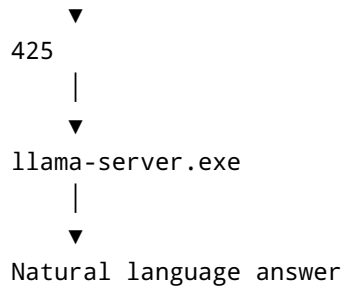
Example

```
multiply(a,b)
```

```
add(a,b)
```

End-to-End Flow





Result

A completely local AI agent capable of:

- Running a local LLM
- Hosting a web UI
- Exposing an OpenAI-compatible REST API
- Discovering MCP tools
- Executing Python tools through MCP
- Returning tool-assisted responses

No cloud services are required.